SC25
St. Louis, MO | hpc ignites.

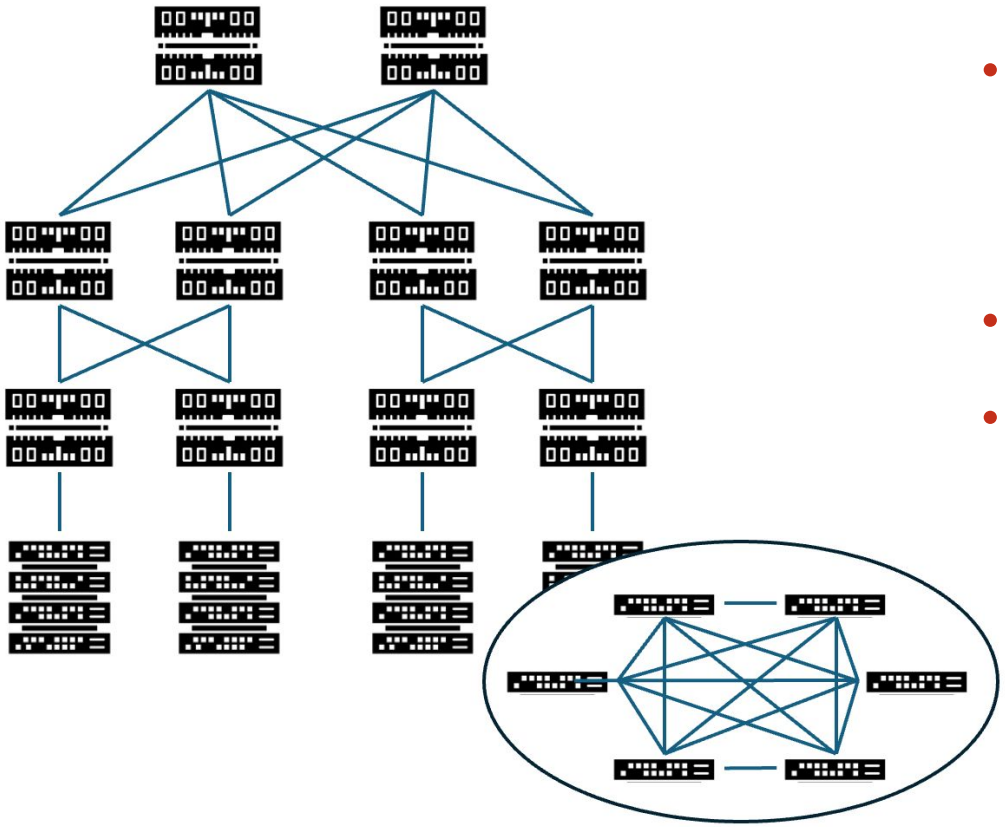
Dynamic Topology-Aware Scheduling in HPC Systems with Topograph

Dmitry Shmulevich, NVIDIA

@whoami

- Cloud Software Engineering at NVIDIA
- Contributor to open source projects
- Maintainer of
  - github.com/NVIDIA/knavigator
  - github.com/NVIDIA/topograph

Problem Statement



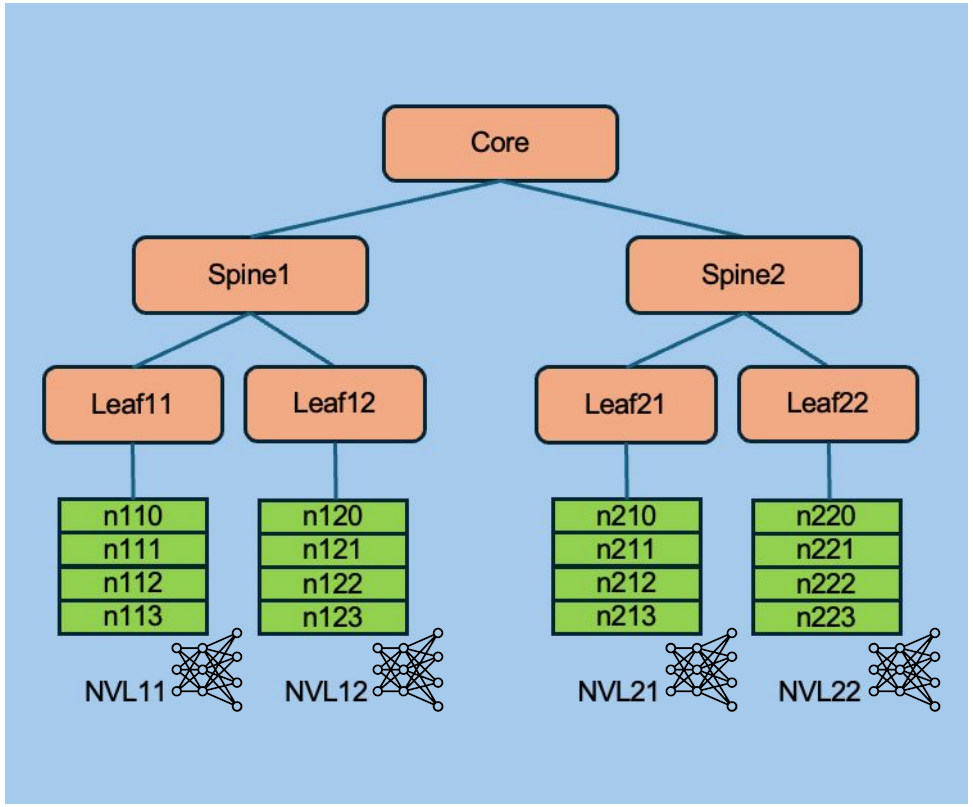
- Rising HPC demands: complexity, scale, resources
- Performance improves on network-optimized nodes
  - InfiniBand XDR: 800 GB/s (bidirectional)
  - NVLink 5.0: 1.8 TB/s (bidirectional)
- SLURM supports topology-aware scheduling
- Challenges:
  - Dynamic environment in large clusters
  - Limited control in cloud environments

Topograph

- Open source
  - https://github.com/NVIDIA/topograph
- Discover and maintain real-time cluster network topology in CSPs and on-prem
- Support
  - SLURM, Kubernetes, SLURM-on-Kubernetes
  - Cloud: AWS, OCI, GCP, Nebius
  - On-prem: InfiniBand, Spectrum-X
  - DRA-enabled Kubernetes deployments

SLURM/Slinky: topology.conf

- Generate cluster-wide topology config
  - topology/tree
  - topology/block

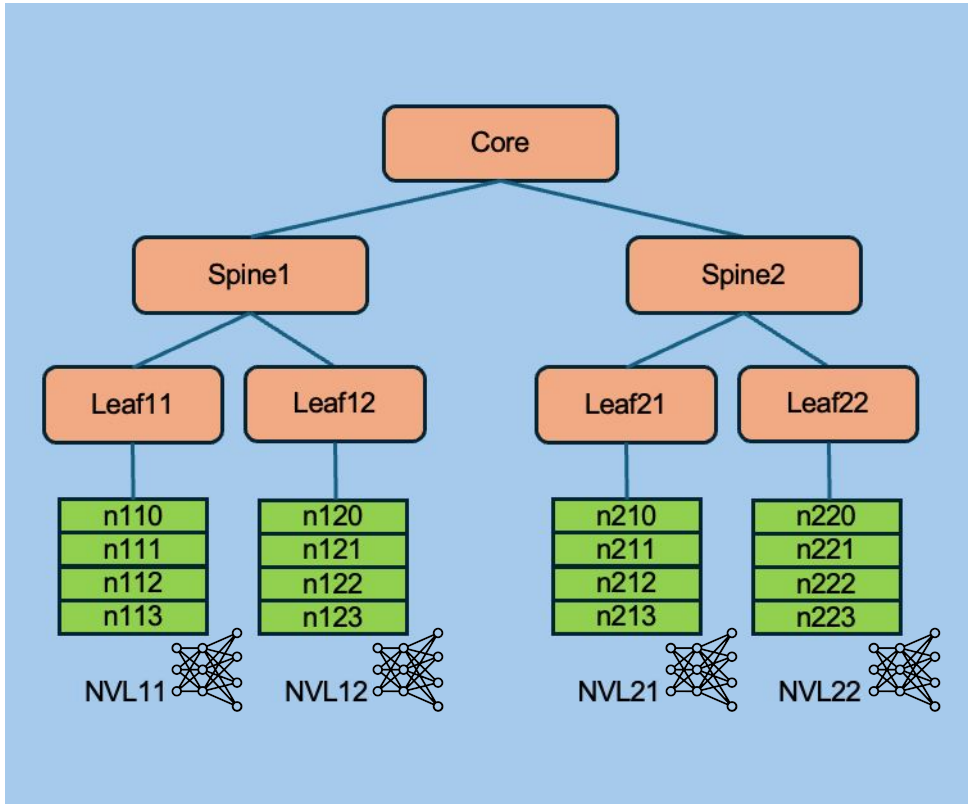



```
#################################################################
# Slurm's network topology configuration file for use with the
# topology/tree plugin
#################################################################
SwitchName=core Switches=spine[1-2]
SwitchName=spine1 Switches=leaf[11-12]
SwitchName=spine2 Switches=leaf[21-22]
SwitchName=leaf11 Nodes=n[110-113]
SwitchName=leaf12 Nodes=n[120-123]
SwitchName=leaf21 Nodes=n[210-213]
SwitchName=leaf22 Nodes=n[220-223]
```

```
#################################################################
# Slurm's network topology configuration file for use with the
# topology/block plugin
#################################################################
# block001=nvl11
BlockName=block001 Nodes=n[110-113]
# block002=nvl12
BlockName=block002 Nodes=n[120-123]
# block003=nvl21
BlockName=block003 Nodes=n[210-213]
# block004=nvl22
BlockName=block004 Nodes=n[220-223]
BlockSizes=4,8,16
```

SLURM/Slinky: topology.yaml

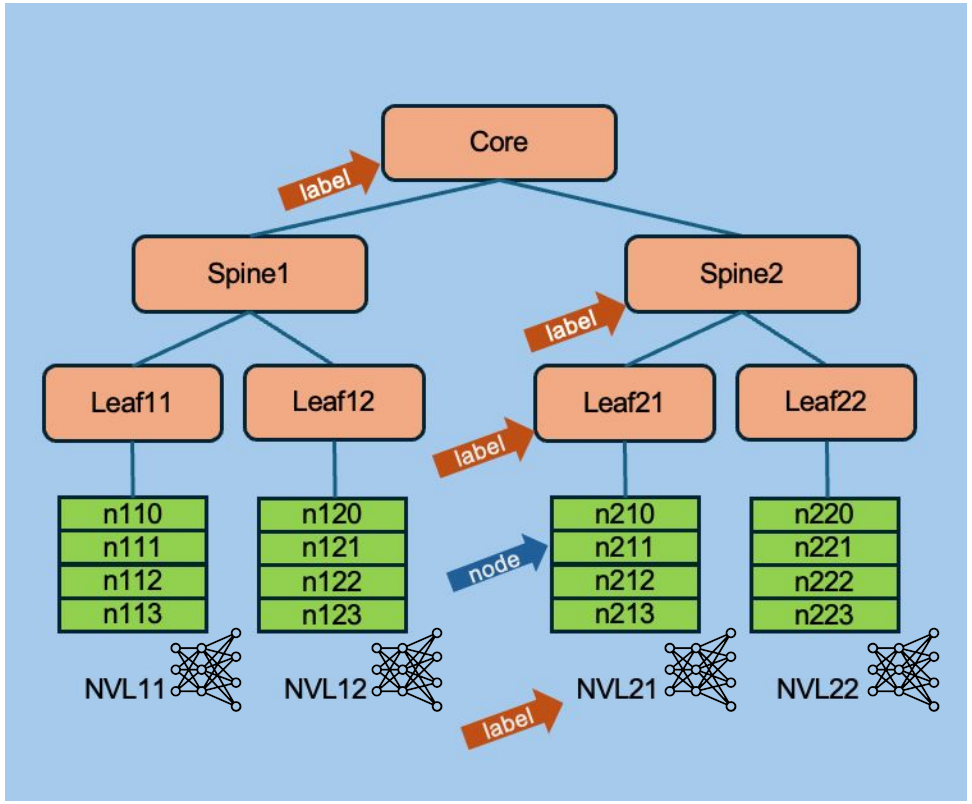
- Generate per-partition topology config
  - SLURM v25.05+




```yaml
- topology: topo1
  clusterDefault: false
  block:
    blockSizes: [4, 8]
    blocks:
    - block: block0
      nodes: n[110-113,120-123]
- topology: topo2
  clusterDefault: false
  block:
    blockSizes: [4]
    blocks:
    - block: block0
      nodes: n[210-213]
- topology: topo3
  clusterDefault: false
  tree:
    switches:
    - switch: core
      children: spine2
    - switch: spine2
      children: leaf22
    - switch: leaf22
      nodes: n[220-223]
```

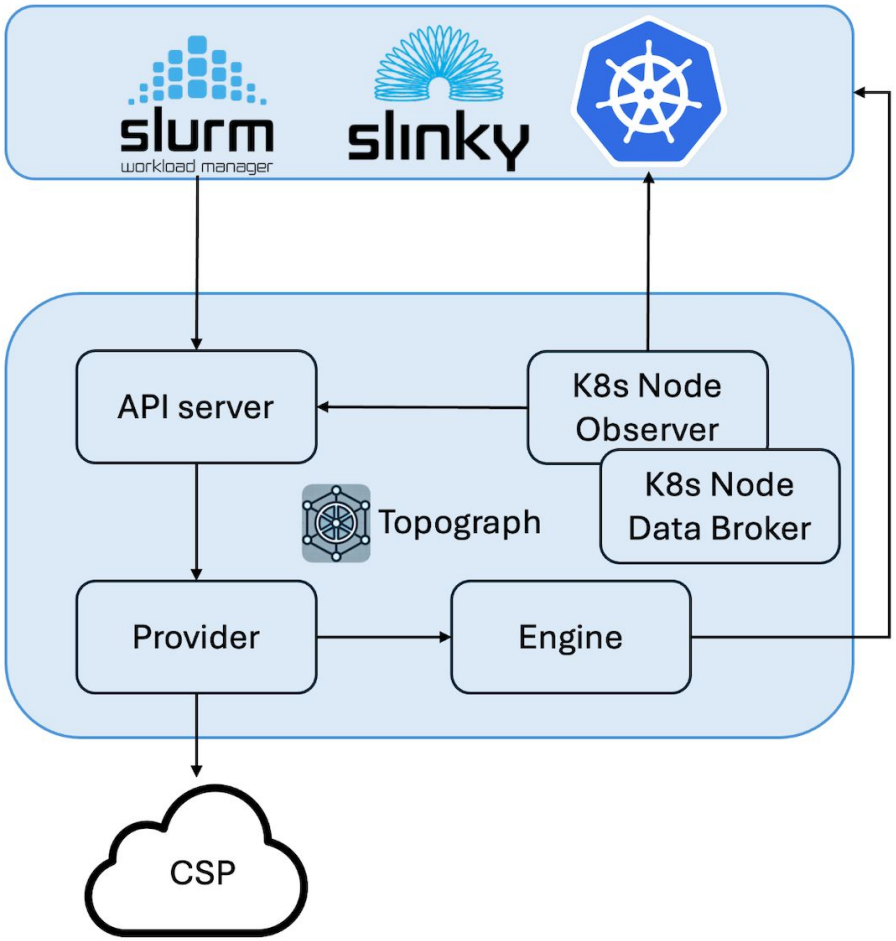
Topograph in Kubernetes

- Apply cluster-wide topology node labels




```yaml
apiVersion: v1
kind: Node
metadata:
  labels:
    kubernetes.io/arch: amd64
    kubernetes.io/hostname: n211
    kubernetes.io/os: linux
    kubernetes.io/role: agent
    network.topology.nvidia.com/accelerator: nvl21
    network.topology.nvidia.com/block: leaf21
    network.topology.nvidia.com/spine: spine2
    network.topology.nvidia.com/datacenter: core
    nvidia.com/gpu.count: "8"
  name: n211
```

Topograph Design



Node Data Broker

- Sets annotations in cluster nodes

Node Observer

- Notifies Topograph API server about node changes

API Server

- Exposes REST API endpoint
- Handles topology requests

Provider

- Interfaces with CSP API or on-prem tools
- Retrieves cluster network topology and converts it into an internal representation

Engine

- Translates network topology into expected format

Usage: SLURM

- Installation via Debian/RPM package
- Topology discovery based on:
  - manual topology request
    - curl -H "Content-Type: application/json" -d @payload.json \ http://localhost:49021/v1/generate
  - automation, e.g. strigger
    - scripts/create-topology-update-script.sh
- Request payload parameters
  - provider: topology provider
  - plugin: tree/block/flat*
  - topologyConfigPath: file path for topology configuration
  - reconfigure: calls scontrol reconfigure to reload SLURM config

```json
{
  "provider": {
    "name": "aws"
  },
  "engine": {
    "name": "slurm",
    "params":{
      "plugin": "topology/block",
      "topologyConfigPath": "/etc/slurm/topology.conf",
      "reconfigure": true
    }
  }
}
```

```json
{
  "provider": {
    "name": "aws"
  },
  "engine": {
    "name": "slurm",
    "params":{
      "topologies": {
        "topo1": {
          "partition": "part1",
          "plugin": "topology/block",
          "blockSizes": [8, 16]
        },
        "topo2": {
          "partition": "part2",
          "plugin": "topology/tree"
        },
        "topo-default": {
          "plugin": "topology/flat",
          "clusterDefault": true
        }
      },
      "topologyConfigPath": "/etc/slurm/topology.conf",
      "reconfigure": true
    }
  }
}
```

Usage: Kubernetes, Slinky

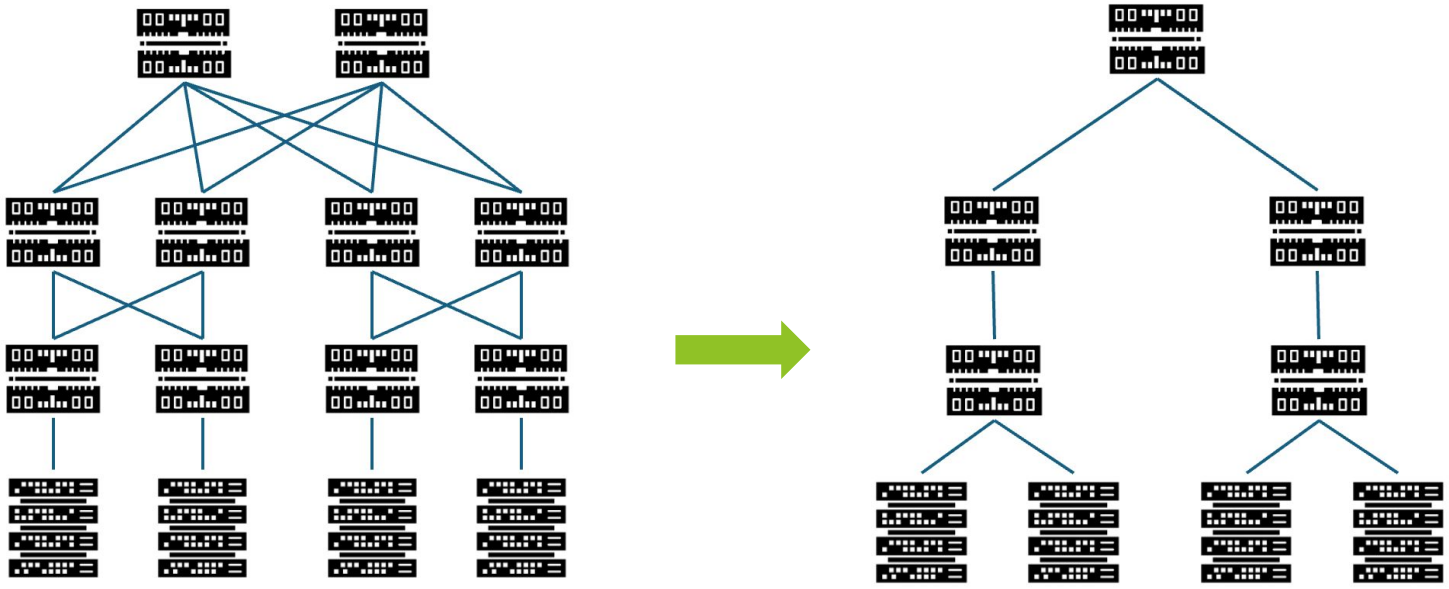
- Installation via Helm chart
- Topology discovery based on changes in
  - slurmd pods (Slinky)
  - cluster nodes (Kubernetes)

```yaml
global:
  provider: oci
  engine: k8s

node-observer:
  topograph:
    trigger:
      nodeSelector: {}
```

```yaml
global:
  provider: oci
  engine: slinky
  engineParams:
    namespace: slurm
    podSelector:
      matchLabels:
        app.kubernetes.io/component: compute
    plugin: topology/tree
    topologyConfigPath: topology.conf
    topologyConfigmapName: slurm-config

node-observer:
  topograph:
    trigger:
      podSelector:
        matchLabels:
          app.kubernetes.io/component: compute
```

Topology Graph Reduction

- CSP APIs typically return a 3-tier tree topology
- On-prem clusters often use a Clos network topology
- Topograph transforms Clos to optimized 3-tier structure
  - Lossless, simplified representation
  - Faster traversal of topology graph and improved scheduler performance



Benefits

- Improved network and GPU utilization
- Reduced execution time and operational cost
- Reduced inter-job interference
- Reduced network latency
- Optimized GPU and memory access

Thank you!